

1 Megoldás

A megoldás három részfeladatot kezel a c paraméter alapján.

1.1 Case 1: Geometriai sorozat keresése hármásával

Ha $c == 1$, akkor meg kell számolnunk, hogy hány olyan i index létezik, amelyre teljesül:

$$v_i \cdot v_{i+1} = v_{i+2}$$

Azaz három egymást követő elem geometriai progressziót alkot.

A megoldás triviális: végigiterálunk az összes i indexen ($1 \leq i \leq n-2$), és megszámoljuk a feltételt kielégítő pozíciókat. Az időkomplexitás $O(n)$.

1.2 Case 2 és 3: Közös struktúra

Mindkét esetben hasonló feladatot kell megoldani: adott egy x érték, keressük meg a legkorábbi pozíciójú olyan (i, j) párt, ahol $v_i \cdot v_j = x$ és $i < j < \text{aktuális pozíció}$.

1.2.1 Kulcs észrevétel: Pozíciótárolás két utolsó előfordulással

Két tömböt használunk:

- $\text{fr}[v]$ — a legutolsó (legnagyobb) pozíció, ahol a v érték előfordult
- $\text{fr2}[v]$ — az azt megelőző pozíció, ahol a v érték előfordult

Amikor egy új x értékkel találkozunk:

```
fr2[v[x]] = fr[v[x]];
fr[v[x]] = x;
```

Ez $O(1)$ idő alatt frissíti a struktúrát.

1.2.2 Osztópárok keresése

Ha tudjuk, hogy $v_i \cdot v_j = X$, akkor v_i és v_j az X osztópárjai. Keressük az összes d osztót 1 -től $\sqrt{X}$ -ig:

$$\text{ha } X \bmod d == 0 : \quad p_1 = d, \quad p_2 = X/d$$

Ha mindkét osztó szerepel a korábbiak között (azaz $\text{fr}[p] > 0$), akkor a két pozíció közül a kisebbet választjuk, és a maximumát tartjuk nyilván:

$$\text{ans} = \max(\text{ans}, \min(\text{fr}[p_1], \text{fr}[p_2]))$$

Ha $p_1 == p_2$ (azaz X négyzetszám), akkor csak a fr2 tartalmaz érvényes második előfordulást.

Az osztókeresés $\sqrt{X}$ időt vesz igénybe.

1.3 Case 2 részletes megoldása

A 2-es esetben csak a végső v_n értékre kell választ adni. Az algoritmus:

1. Beolvassa a teljes tömböt
2. Végigfut a tömbön, közben tárolva a **fr** és **fr2** értékeket
3. Végül megvizsgálja v_n összes osztópárját

Az eredmény a legnagyobb $\min(\text{pozíció}_1, \text{pozíció}_2)$ érték.

1.4 Case 3 részletes megoldása

A 3-as esetben minden pozícióra meg kell adni a választ, és összegezni kell őket. Az algoritmus:

1. Minden x pozícióra (1-től n -ig):
 - (a) Kiszámítja a helyi választ a fenti módszerrel
 - (b) Hozzáadja az összeghez
 - (c) Frissíti **fr** és **fr2**-t a jelenlegi $v[x]$ értékkel

Fontos megjegyzés: A számítás **mielőtt** történik meg, hogy $v[x]$ a tömbbe kerülne, így a **fr** és **fr2** csak a korábbi elemeket tartalmazza.

1.5 Összefoglalás

- **1. eset:** $O(n)$ idő, $O(1)$ memória
- **2. eset:** $O(n + \sqrt{v_n})$ idő, $O(n)$ memória
- **3. eset:** $O(n + \sum \sqrt{v_i})$ idő, $O(n)$ memória

A megoldás lényege, hogy a pozíciók tárolásával és az osztópárok keresésével hatékonyan megválaszolható a geometriai progressziókkal kapcsolatos kérdés.